

IOT 2025/2026 - Challenge 2

PART 1: packet sniffing

Simone Amighini (person code: 10832147)

CQ1)

a)

I have filtered all the non confirmable delete requests to *coap.me* server and all the successful delete responses from it:

ip.addr == 134.102.218.18 and ((coap.type == 1 and coap.code == 4) or (coap.type == 1 and coap.code == 66))

There is only one **request** that has a successful **response**.

Answer: MID = 30800

No.	Time	Source	Destination	Protocol	Length	Info
8442	297.091099803	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:63788, DELETE, TKN:07 f2 0b 6a d0 47 d7 49, /5
212	57.880374744	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:63041, DELETE, TKN:f3 5b bf 03 a5 ff 38 cc, /3
8524	301.042096249	10.0.2.15	134.102.218.18	CoAP	61	NON, MID:62535, DELETE, TKN:73 70 51 f7 9b a1 e4 d2, /seg1
2007	101.712911188	10.0.2.15	134.102.218.18	CoAP	66	NON, MID:57936, DELETE, TKN:e9 3d bb cb 6a 59 8a 7d, /location1
96	36.776462558	10.0.2.15	134.102.218.18	CoAP	63	NON, MID:53878, DELETE, TKN:03 0b da 63 ff 31 59 2a, /broken
9250	318.124411487	10.0.2.15	134.102.218.18	CoAP	63	NON, MID:49932, DELETE, TKN:45 2a 18 97 9a 58 70 44, /secret
8447	297.100757882	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:47220, DELETE, TKN:1c 60 83 b6 96 fb 32 31, /3
3372	166.834428061	10.0.2.15	134.102.218.18	CoAP	63	NON, MID:38480, DELETE, TKN:57 f9 87 c1 79 a3 2d 78, /secret
342	62.723048717	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:38052, DELETE, TKN:23 01 e4 de 41 56 74 9f, /weird333
44	18.469208769	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:33264, DELETE, TKN:7b 93 76 a6 a9 48 4b 8b, /5
2260	111.791750738	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:32298, DELETE, TKN:73 59 ba 94 93 b3 9c 78, /4
3597	178.968919054	10.0.2.15	134.102.218.18	CoAP	61	NON, MID:32242, DELETE, TKN:78 01 af 5e a0 b3 48 f0, /seg1
11075	428.150151063	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:30800, DELETE, TKN:88 0f 30 d8 b5 43 d6 f7, /validate
4982	242.914933760	10.0.2.15	134.102.218.18	CoAP	61	NON, MID:30212, DELETE, TKN:25 2c f4 36 b7 ef a3 ea, /path
11078	428.332370619	134.102.218.18	10.0.2.15	CoAP	67	NON, MID:26010, 2.02 Deleted, TKN:88 0f 30 d8 b5 43 d6 f7, /validate (text/plain)
2121	107.789056001	10.0.2.15	134.102.218.18	CoAP	62	NON, MID:25814, DELETE, TKN:8b 6f 78 cb e0 76 f9 16, /hello
2912	152.878450073	10.0.2.15	134.102.218.18	CoAP	61	NON, MID:24716, DELETE, TKN:e0 86 49 98 dd 73 82 06, /seg1
7807	285.013210552	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:22836, DELETE, TKN:7f 09 e5 67 7c bb cb 42, /4
11923	468.193163910	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:21028, DELETE, TKN:75 91 2d 3b 51 bc 50 16, /3
5379	265.139760939	10.0.2.15	134.102.218.18	CoAP	58	NON, MID:11787, DELETE, TKN:21 a5 87 c5 a0 ac 8a 6c, /3
2206	108.735128949	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:1059, DELETE, TKN:b6 51 a7 4c 2a 74 ae cd, /separate

b)

I have filtered all the requests to the /validate resource:

coap.opt.uri_path == "/validate"

No.	Time	Source	Destination	Protocol	Length	Info
45	18.469227361	10.0.2.15	134.102.218.18	CoAP	65	CON, MID:56172, PUT, TKN:93 a9 3f d7 11 c5 65 f2, /validate
145	54.764186215	10.0.2.15	134.102.218.18	CoAP	65	CON, MID:41030, POST, TKN:d8 d8 05 54 80 35 5c e9, /validate
535	68.822751780	10.0.2.15	134.102.218.18	CoAP	65	CON, MID:38569, POST, TKN:53 cc 36 96 83 89 46 13, /validate
2530	127.726031905	10.0.2.15	134.102.218.18	CoAP	65	CON, MID:47391, DELETE, TKN:b7 8a a8 21 b9 c6 5d 66, /validate
2765	142.755108470	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:6160, PUT, TKN:48 f2 cb 8f 31 31 52 d3, /validate
5050	246.900913430	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:34025, PUT, TKN:bf ae fb b9 49 69 27 e5, /validate

5291	258.973837622	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:15213, PUT, TKN:c9 79 d3 6b dc 8b 00 a9, /validate
11075	428.150151063	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:30800, DELETE, TKN:88 0f 30 d8 b5 43 d6 f7, /validate
11363	445.398432889	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:5096, GET, TKN:38 f2 48 20 7c f5 0a 87, /validate

The **response** to the **last GET request** is shown using:

coap.token == 38f248207ef50a87

11363	445.398432889	10.0.2.15	134.102.218.18	CoAP	65	NON, MID:5096, GET, TKN:38 f2 48 20 7c f5 0a 87, /validate
11364	445.488827848	134.102.218.18	10.0.2.15	CoAP	71	NON, MID:41395, 2.05 Content, TKN:38 f2 48 20 7c f5 0a 87, /validate (text/plain)

The only deletion found in point a) is actually not successful because a following GET request returns a valid value and no insertion is between the two requests.

Answer: 0

CQ2)

Firstly, I have used a shell command to filter all the unsuccessful responses sent from the local CoAP server and store them as a comma separated list inside a variable:

RESPONSES_MID=\$(tshark -r A.pcapng -Y "(ip.src == 127.0.0.1 or ipv6.src == ::1) and coap.type == 2 and coap.code >= 128" -T fields -e coap.mid | sort | uniq | paste -sd, -)

Then, I have used another command to filter all the POST and PUT requests associated to such responses:

tshark -r A.pcapng -Y "(ip.dst == 127.0.0.1 or ipv6.dst == ::1) and coap.type == 0 and coap.code in {2, 3} and coap.mid in {\$RESPONSES_MID}" -T fields -E header=y -e udp.srcport -e coap.code -e coap.opt.uri_path

The result is the following (note that coap.code = 2 means POST while coap.code = 3 means PUT):

udp.srcport	coap.code	coap.opt.uri_path
42322	3	main_door
49142	3	dining_room,temperature
52701	2	test
34331	2	dining_room,temperature
49136	2	dining_room,temperature
56263	3	basic
37726	2	dining_room,temperature
41093	3	dining_room,door
37754	3	living_room,door
49994	2	hello_world
59723	2	test
42690	3	main_door
54652	2	dining_room
44649	2	basic
50549	2	basic
60982	2	main_door
41536	3	dining_room
57529	2	living_room,temperature
43306	2	living_room

44135	2	dining_room,temperature
44135	2	dining_room,temperature
42952	2	hello_world
41282	2	dining_room,door
44481	2	dining_room,door
55898	3	dining_room,temperature
57761	2	hello_world
45140	2	hello_world

Finally, I have counted by hand how many **unique (which I intended as counting multiple requests for the same method starting from the same port as one request)** POST and PUT requests have been made for each resource:

Resource	Unique POST requests	Unique PUT requests
/main_door	1	2
/dining_room/temperature	4	2
/test	2	0
/basic	2	1
/dining_room/door	2	1
/living_room/door	0	1
/hello_world	4	0
/dining_room	1	1
/living_room/temperature	1	0
/living_room	1	0

Only /dining_room satisfies the requested condition.

Answer: 1

CQ3)

a)

I have filtered the observe requests using:

coap.opt.observe == 0

No.	Time	Source	Destination	Protocol	Length	Info
11136	430.278633357	127.0.0.1	127.0.0.1	CoAP	77	CON, MID:12425, GET, TKN:1e a4, Block #0, /living_room/temperature
4343	214.773794714	127.0.0.1	127.0.0.1	CoAP	77	CON, MID:10588, GET, TKN:89 b9, Block #0, /dining_room/temperature

Then, I have noticed that the observe request regarding /dining_room/temperature has its token equal to 89b9 so I have filtered the entire relative session using:

coap.token == 89b9

No.	Time	Source	Destination	Protocol	Length	Info
4343	214.773794714	127.0.0.1	127.0.0.1	CoAP	77	CON, MID:10588, GET, TKN:89 b9, Block #0, /dining_room/temperature
4344	214.777777148	127.0.0.1	127.0.0.1	CoAP/JSON	117	ACK, MID:10588, 2.05 Content, TKN:89 b9, End of Block #0, /dining_room/temperature, JSON (application/json)
4620	222.532566824	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:35, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
5098	249.494821853	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:38, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
5623	273.825360943	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:43, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
8509	300.820865762	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:45, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
8414	326.429577051	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:49, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
9866	350.769651227	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)

9867	350.769668653	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
9937	353.696910139	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
9938	353.696940679	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10001	359.552073924	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10002	359.552088425	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10193	371.257323429	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10194	371.257333119	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10573	394.669233647	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:62, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
10574	394.669244742	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10631	397.475501427	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:62, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10632	397.475510907	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11220	436.761538592	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:74, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
11221	436.761561322	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11917	467.521064665	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:87, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
11918	467.521073170	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
12782	498.166885839	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:90, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
12783	498.166901834	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)

By looking at the observe sequence number in the responses, I have found that 10 separate notifications are sent.

Answer: 10

b)

Looking at the responses found in point a), the last 5 are useless because they have not reached their destination, as stated by an ICMP port unreachable message indicating that the client is no more listening on UDP port 48049.

IMPORTANT NOTE: for “useless” response I have intended an undelivered response.

“No.	Time	Source	Destination	Protocol	Length	Info
4343	214.773794714	127.0.0.1	127.0.0.1	CoAP	77	CON, MID:10588, GET, TKN:89 b9, Block #0, /dining_room/temperature
4344	214.777777148	127.0.0.1	127.0.0.1	CoAP/JSON	117	ACK, MID:10588, 2.05 Content, TKN:89 b9, End of Block #0, /dining_room/temperature, JSON (application/json)
4620	222.532566824	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:35, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
5098	249.494821853	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:38, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
5623	273.825360943	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:43, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
8509	300.820865762	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:45, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
9414	326.429577051	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:49, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
9866	350.769651227	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
9867	350.769668653	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
9937	353.696910139	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
9938	353.696940679	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10001	359.552073924	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10002	359.552088425	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10193	371.257323429	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:54, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10194	371.257333119	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10573	394.669233647	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:62, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
10574	394.669244742	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
10631	397.475501427	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:62, 2.05 Content, TKN:89 b9, /dining_room/temperature [Retransmission], JSON (application/json)
10632	397.475510907	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11220	436.761538592	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:74, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
11221	436.761561322	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
11917	467.521064665	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:87, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
11918	467.521073170	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)
12782	498.166885839	127.0.0.1	127.0.0.1	CoAP/JSON	115	CON, MID:90, 2.05 Content, TKN:89 b9, /dining_room/temperature, JSON (application/json)
12783	498.166901834	127.0.0.1	127.0.0.1	ICMP	143	Destination unreachable (Port unreachable)

Answer: 5

CQ4)

I have set the default port for MQTTSN to 1885 in Wireshark settings, then I have filtered all the MQTT-SN packets sent by the local broker from port 1885 using:

mqttsn and ip.src == 127.0.0.1 and udp.srcport == 1885

Answer: 0

CQ5)

Firstly, I have filtered all the connect requests that have the last will flag active:

mqtt.conflog.willflag == 1

No.	Time	Source	Destination	Protocol	Will Message	Will Topic
4	0.000117188	::1	::1	MQTT	6572726f723a20612056495020436c69656e74206a7573742064696564	university/department12/room1/temperature
196	2.116585177	10.0.2.15	5.196.78.28	MQTT	6572726f723a20706575697664716c	metaverse/room2/floor4
352	5.034840089	10.0.2.15	5.196.78.28	MQTT	6572726f723a20786c7a6171707464	hospital/facility3/area3
557	7.043177949	10.0.2.15	5.196.78.28	MQTT	6572726f723a207a6a7a7772636470	metaverse/room2/room2

Then I have filtered all the last will messages that have been sent:

mqtt.msg in {6572726f723a20612056495020436c69656e74206a7573742064696564, 6572726f723a20706575697664716c, 6572726f723a20786c7a6171707464, 6572726f723a207a6a7a7772636470}

No.	Time	Source	Destination	Protocol	Length	Info
6562	146.692187516	::1	::1	MQTT	164	Publish Message (id=1) [university/department12/room1/temperature]
6566	146.692209983	::1	::1	MQTT	164	Publish Message (id=1) [university/department12/room1/temperature]
6564	146.692199911	::1	::1	MQTT	165	Publish Message (id=12) [university/department12/room1/temperature]
6560	146.692096889	::1	::1	MQTT	162	Publish Message [university/department12/room1/temperature]

Given that all the last will messages that have been sent involve the topic *university/department12/room1/temperature* and have the localhost broker as sender, I filtered all the subscriptions to the localhost broker containing wildcards and the word “university” using:

ipv6.dst == ::1 and tcp.dstport == 1883 and (mqtt.topic contains "#" or mqtt.topic contains "+") and mqtt.topic contains "university"

No.	Time	Source	Destination	Protocol	Length	Info
226	3.087146589	::1	::1	MQTT	117	Subscribe Request (id=1) [university/+/+/deposit]
3612	25.177604306	::1	::1	MQTT	126	Subscribe Request (id=12) [university/building2/section0#]
3119	18.136764543	::1	::1	MQTT	119	Subscribe Request (id=12) [university/department3/+/+]
3276	20.152592426	::1	::1	MQTT	119	Subscribe Request (id=14) [university/department3/#]
3483	23.220257088	::1	::1	MQTT	113	Subscribe Request (id=14) [university/room0#]
2330	13.118621965	::1	::1	MQTT	119	Subscribe Request (id=4) [university/department3/#]
490	6.106824619	::1	::1	MQTT	117	Subscribe Request (id=4) [university/facility0#]
1136	10.102492445	::1	::1	MQTT	108	Subscribe Request (id=6) [university/#]

There is only one subscriber on localhost that receives a last will message due to **its wildcarded subscription *university/#***.

Answer: 1

CQ6)

a)

I have used a shell command to extract all the clients that made erase requests to HiveMQ:

```
tshark -r B.pcapng -Y "ip.dst in {18.192.151.104, 35.158.43.69} and tcp.dstport == 1883 and mqtt.retain == 1 and mqtt.msg == \"\" -T fields -e ip.src -e tcp.srcport | sort | uniq
```

```
10.0.2.15 34779
10.0.2.15 36777
10.0.2.15 38207
10.0.2.15 47753
10.0.2.15 49877
```

Answer: 5

b)

I have filtered the clients found in point a) that have used a clientid > 7 bytes:

```
ip.src == 10.0.2.15 and tcp.srcport in {34779, 36777, 38207, 47753, 49877} and len(mqtt.clientid) > 7
```

No.	Time	Source	Destination	Protocol	Length	Info
553	7.033325651	10.0.2.15	18.192.151.104	MQTT	80	Connect Command
1752	11.072194429	10.0.2.15	18.192.151.104	MQTT	84	Connect Command

Answer: 2

CQ7)

I have filtered all the subscriptions to the local broker and then used a regular expression to find the ones that contain at least two wildcards:

```
(ipv6.dst == ::1 or ip.dst == 127.0.0.1) and tcp.dstport == 1883 and mqtt.msgtype == 8 and mqtt.topic matches ".*[+#[].*[+#[].*"
```

No.	Time	Source	Destination	Protocol	Length	Info
226	3.087146589	::1	::1	MQTT	117	Subscribe Request (id=1) [university/+/#deposit]
363	5.106636971	::1	::1	MQTT	113	Subscribe Request (id=3) [hospital/+/#light]
460	6.092938713	::1	::1	MQTT	114	Subscribe Request (id=4) [metaverse/+/#room1/#]
473	6.099235260	::1	::1	MQTT	117	Subscribe Request (id=3) [house/department3/+/#]
700	9.110386346	::1	::1	MQTT	118	Subscribe Request (id=6) [metaverse/facility0/+/#]
1177	10.112498473	::1	::1	MQTT	114	Subscribe Request (id=7) [hospital/fchugs/+/#]
1915	12.116734489	::1	::1	MQTT	124	Subscribe Request (id=8) [metaverse/+/#hydraulic_valve]
2360	13.118711456	::1	::1	MQTT	106	Subscribe Request (id=8) [factory/+/#]
2994	16.145074080	::1	::1	MQTT	111	Subscribe Request (id=12) [house/+/#room1/#]

Answer: 9

CQ8)

For capture A I have used the following shell command to filter all the PUBLISH messages to the local MQTT broker and obtain the distribution of messages for topic layers:

```
tshark -r A.pcapng -Y "(ip.dst == 127.0.0.1 or ipv6.dst == ::1) and tcp.dstport == 1883 and mqtt.msgtype == 3" -T fields -e mqtt.topic | awk -F'/' '{print NF}' | sort | uniq -c
```

154	2	# 154 PUBLISH messages having a 2 layer topic
131	3	# 131 PUBLISH messages having a 3 layer topic
154	4	# 154 PUBLISH messages having a 4 layer topic

The total number of PUBLISH messages is 439.

For capture B, the command is the same:

```
tshark -r B.pcapng -Y "(ip.dst == 127.0.0.1 or ipv6.dst == ::1) and tcp.dstport == 1883 and mqtts.msgrtype == 3" -T fields -e mqtts.topic | awk -F/ '{print NF}' | sort | uniq -c
```

21	1	# 21 PUBLISH messages having a 1 layer topic
172	2	# 172 PUBLISH messages having a 2 layer topic
162	3	# 162 PUBLISH messages having a 3 layer topic
179	4	# 179 PUBLISH messages having a 4 layer topic

The total number of PUBLISH messages is 534.

Distribution of MQTT topic depth for local broker

